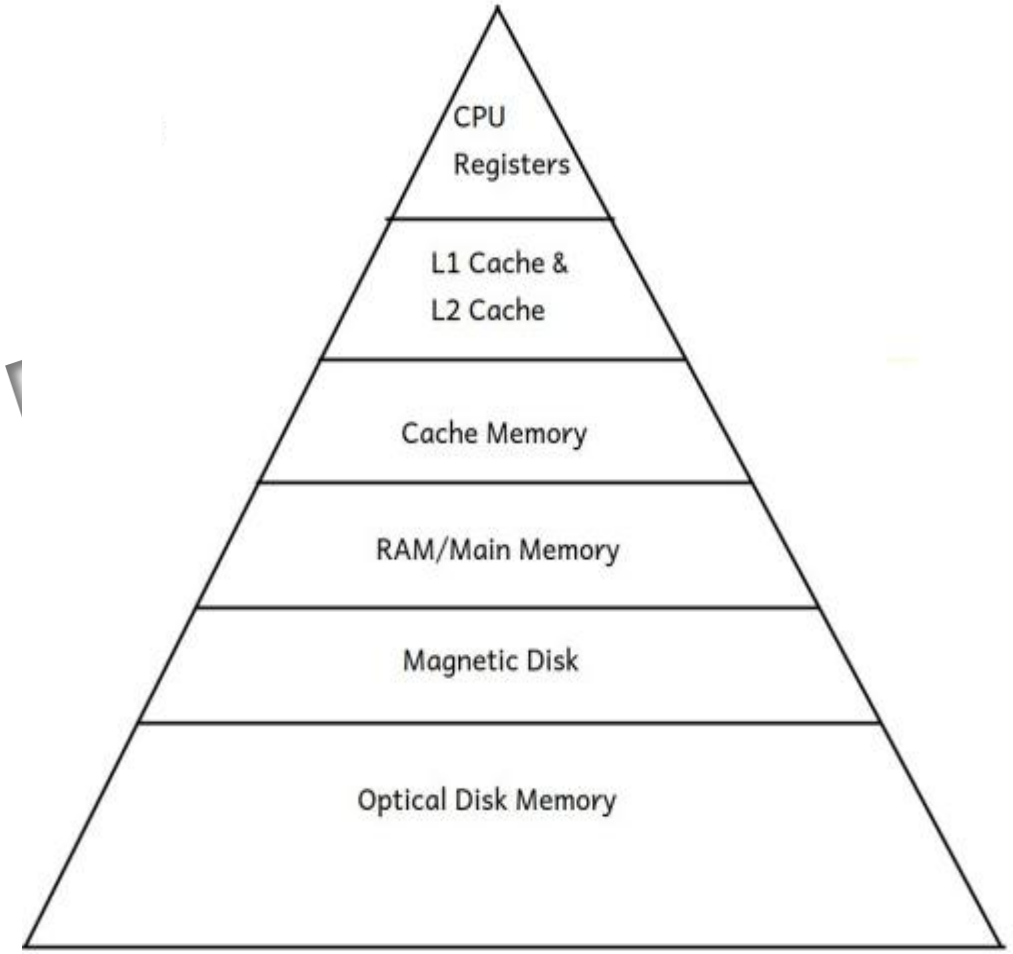
Memory Management

SUNBEAM

➢Computer Memory Technologies

As we go Top to Bottom
1.Acses speed decrease
2.Coast also decrease
3.Capacity increase



Computer Memory Hierarchy

➢ Computer Memory Technologies:

- **CPU Registers:** memory which is very close to the CPU are registers which is at the top in a computer memory hierarchy.
- Instructions and data currently executing by the CPU can be kept temporarily into the CPU registers.

➢ Computer memory can be categorized into two categories as per its location:

- **Internal Memory & External Memory.** - Internal Memory: memory which is internal to the motherboard is referred as an internal memory.
  - e.g. CPU registers, L1 & L2 cache Cache memory, RAM.
- **External Memory:** memory which is external to the motherboard is referred as an external memory.
  - e.g. magnetic disk, optical disk, magnetic tape etc...

SUNBEAM

➢Computer Memory Technologies:

- Computer memory can also categorized into two categories**: Primary Memory & Secondary Memory**.

- **Primary Memory: ** memory which can be accessible directly by the CPU is referred as primary memory, i.e. memory which can accessible by the CPU with the help of instruction set having with the CPU.
  - e.g. CPU registers, L1 & L2 Cache, Cache Memory, RAM
- **Secondary Memory: ** memory which cannot be accessed directly by the CPU is referred as secondary memory.
  - e.g. Magnetic Disk, CD/DVD, PD etc..
  - If the CPU want to access disk contents, first it gets fetched into the RAM and then it can be accessed by the CPU from RAM.

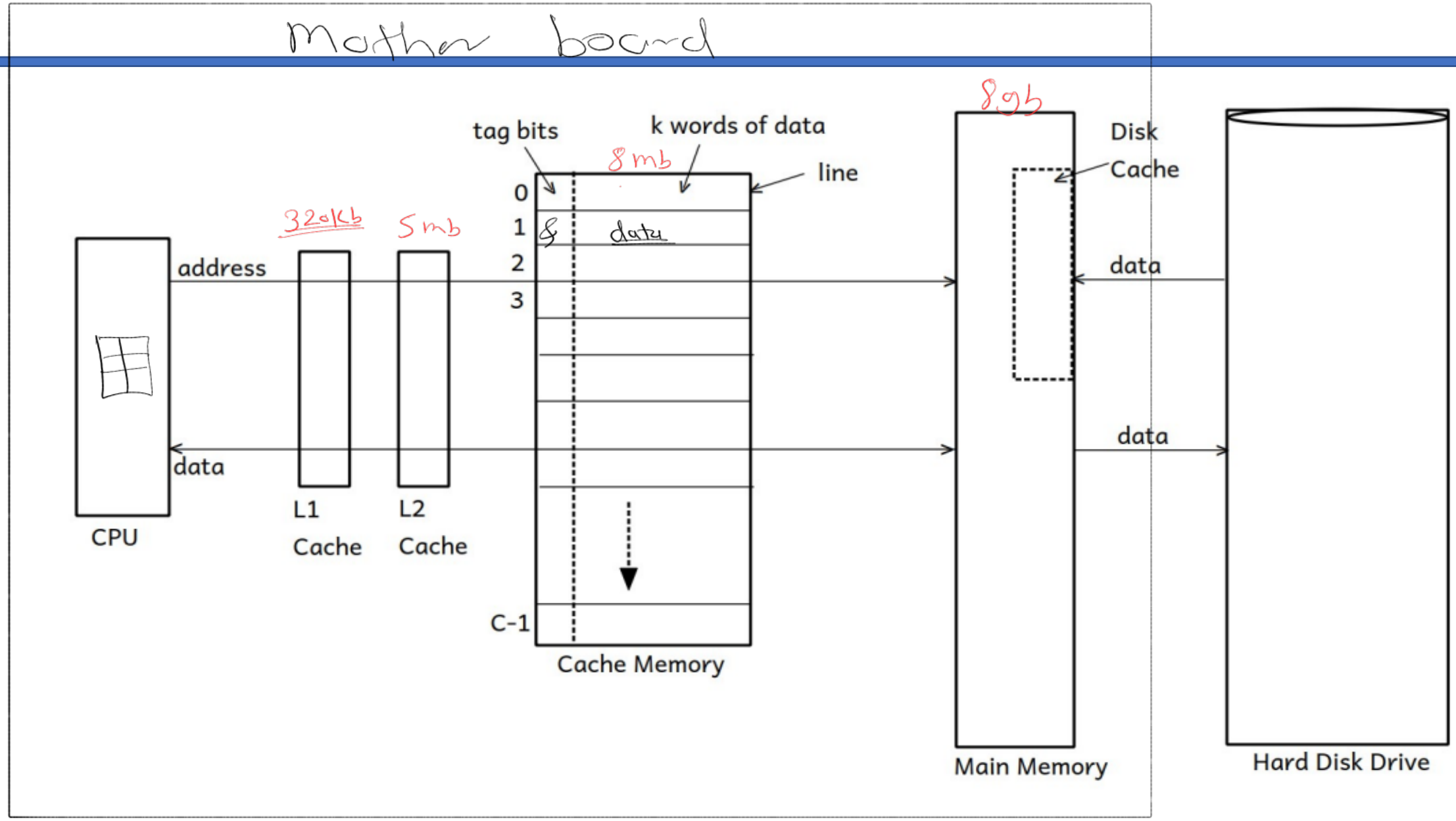
- As for an execution of every program RAM memory is must and hence RAM is also called **as Main memory.**

SUNBEAM

➢ Why there is a need of cache memory?

- As the rate at which the CPU can execute instructions is faster than the rate at which data can be accessed from the main memory, so even the CPU is very fast, with the same speed data do not gets fetched from the main memory for execution, hence due to this speed mismatch overall system performance gets down.
- To reduce speed match between the CPU and the main memory Cache memory (hardware) can be added between them and system performance can be increases by means **reducing speed mismatch.**

SUNBEAM


Mother board
tag bits
k words of data
line
8 mb
0
1
2
3
C-1
data
32 kb
5 mb
address
data
CPU
L1
Cache
L2
Cache
Cache Memory
8 gb
Disk
Cache
data
data
Main Memory
Hard Disk Drive

➢Memory Management

❖. Why there is a need of memory (main memory)management ?

- As main memory is must for an execution of any program and it is a limited memory, hence an OS manages main memory.
- To achieve maximum CPU utilization, an OS must support **multitasking, and to support multi-tasking multiple processes** must be submitted into the system at a time i.e. it must support multiprogramming, but **as main memory is limited** to support multiprogramming an **OS has to do memory management to complete an execution of all submitted processes.**
- **Memory space of one process should gets protected from another process**.

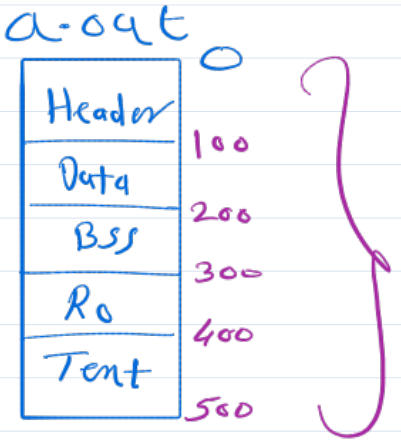
SUNBEAM

- Addresses generated by compiler (i.e. compiler + linker) are referred as **logical addresses.**
- Addresses which can be see by the process when it is in the main memory referred as **physical addresses**.
- **MMU (Memory Management Unit):** which is a hardware unit converts logical address into physical address.
- **MMU is a hardware** contains adder circuit, comparator circuit, base register and limit register. Values of base register and limit registers gets change during context-switch, and memory space **of one process gets protected from another process**.
- CPU always executes program in its **logical memory space**.

SUNBEAM

* Logical Address Space.

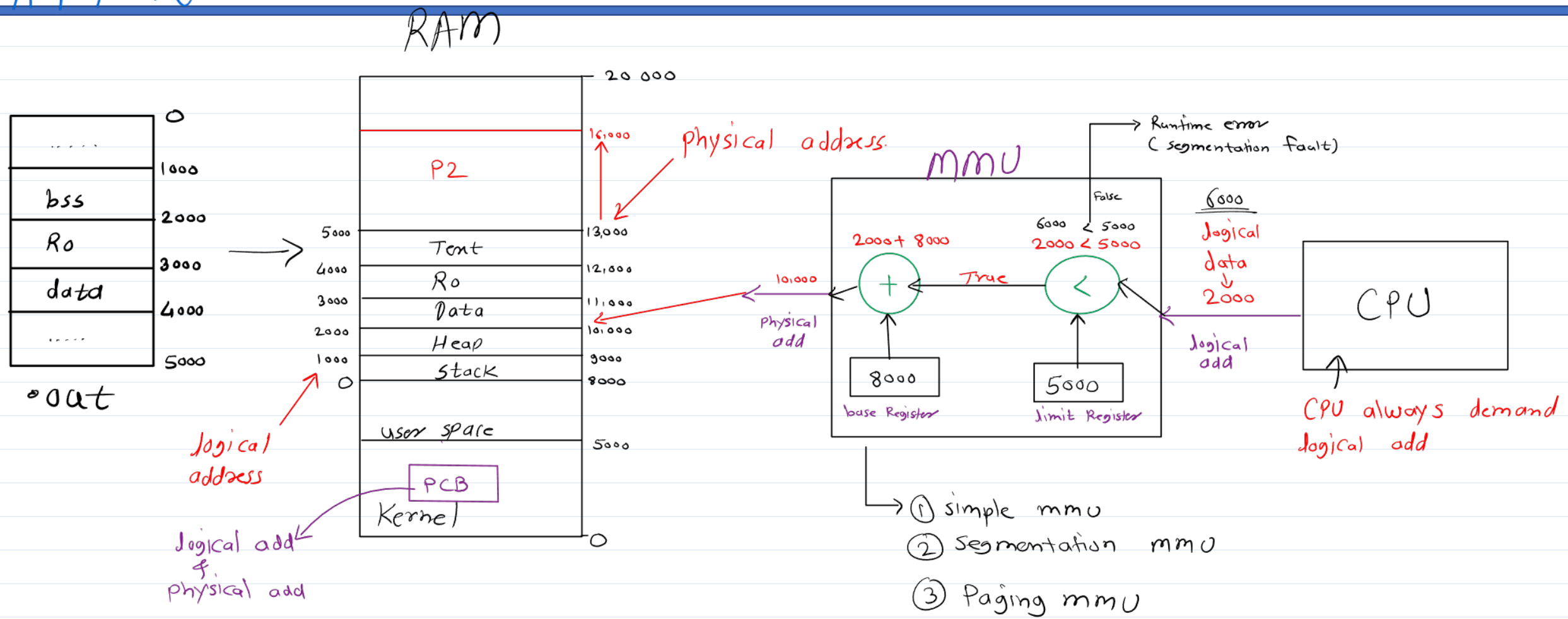
- C → Compiler → .out ⇒



+ linker

logical address space
or
Virtual address space

* MMU


RAM
0
1000
2000
3000
4000
5000
bss
Ro
data
.out
20 000
16,000
13,000
12,000
11,000
10,000
9000
8000
5000
0
P2
Text
Ro
Data
Heap
Stack
user space
PCB
Kernel
physical address
logical address
logical add & physical add
MMU
Runtime error (segmentation fault)
False
6000 < 5000
2000 < 5000
2000 + 8000
True
10,000
physical add
8000
base Register
5000
limit Register
6000
logical data
2000
logical add
CPU
CPU always demand logical add
1) simple mmu
2) Segmentation mmu
3) Paging mmu

➢Memory Allocation:

- When a process is requesting for the main memory, there are two methods by which memory gets allocated for any process

1. Contiguous Memory Allocation

2. Non - contiguous Memory Allocation

SUNBEAM

1. Contiguous Memory Allocation:

- Under this method, process can complete its execution only if memory gets allocated for it in a contiguous manner.
- There are two methods by which memory gets allocated for process under contiguous memory allocation method.

1. Fixed Size Partitioning

2. Variable Size Partitioning

1. Fixed Size Partitioning:

- In this method, physical memory i.e. main memory (user space) is divided into fixed number of partitions and size of each partition is remains fixed.
- If any process is requesting for the memory it can be loaded into main memory in any free partition in which it can be fit.

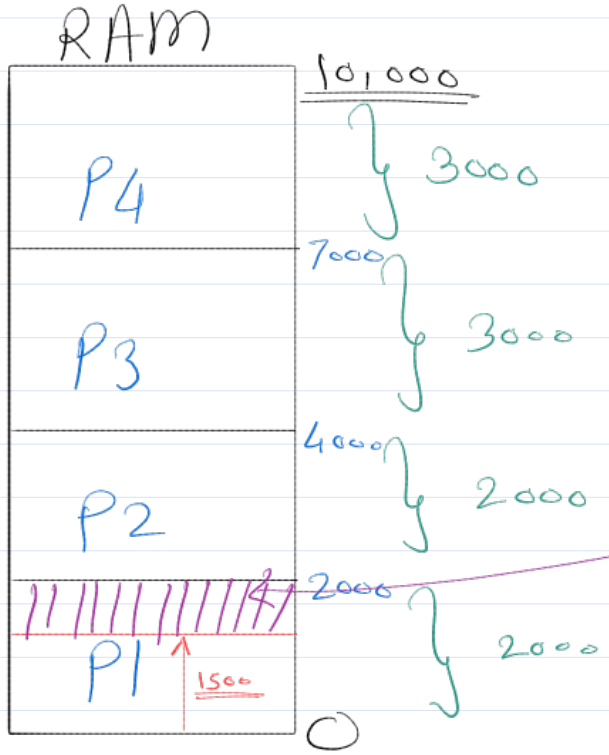
❖Advantages:

- This method is simple to implement.

SUNBEAM

1 Contiguous Allocation

a Fixed size partition.



P6 ⇒ 4000

✓ simple to implement

✗ Degree of M.P. is constant

✗ max size of process is limited

✗ Internal fragmentation.

❖Disadvantages:

- **Internal fragmentation**: if memory remains unused which is internal to the partition.
- **Degree of multi-programming** is limited to the number of partitions in the main memory.
- **Maximum size of a process is limited** to max size partition in the main memory.
- To overcome limitations/disadvantages of fixed size partitioning method, variable/dynamic size partitioning method has been designed.

SUNBEAM

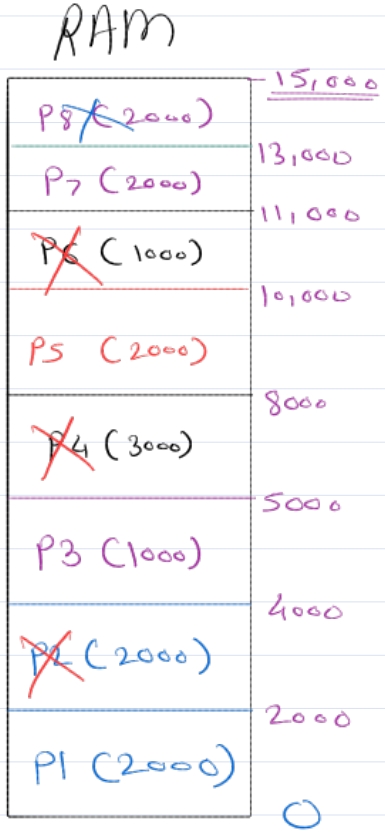
2. Variable/Dynamic Size Partitioning:

- In this method, initially whole user space i.e. physical memory is considered as a single free partition, and processes gets loaded into the main memory as they request for it.
- Size of partition and number of partitions are not fixed in advanced, it gets decided dynamically.
- Memory is allocated to each process as per its availability in the RAM.
- After **allocation** and **deallocation** of few processes, RAM will have few used slots and few free slots.
- OS keep track of free slots in form of a table. For any new process, OS use one of the following mechanism to allocate the free slot.
  - **First Fit**: Allocate first free slot which can accommodate the process.
  - **Best Fit**: Allocate that free slot to the process in which minimum free space will remain.
  - **Worst Fit**: Allocate that free slot to the process in which maximum free space will remain.
- Statistically it is proven that First fit is faster algo; while best fit provides better memory utilization.
- Memory info (physical base address and size) of each process is stored in its PCB and will be loaded into MMU registers (base & limit) during context switch.
- CPU request virtual address (address of the process) and is converted into physical address by MMU as shown in diag.
- If invalid virtual address is requested by the CPU, process will be terminated.

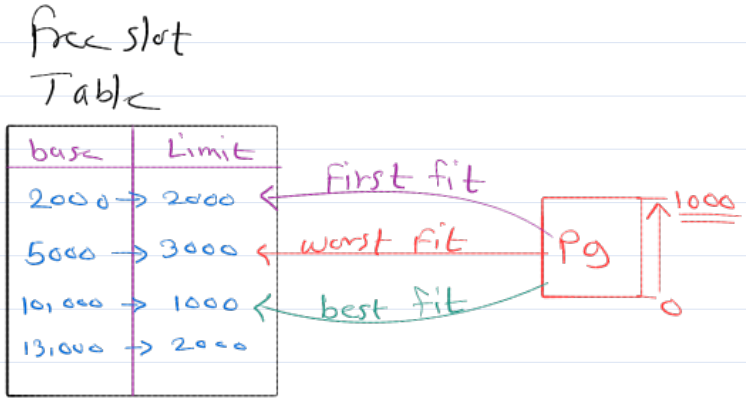
SUNBEAM

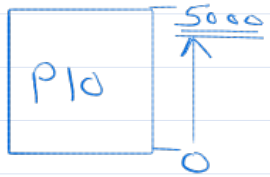
1 Contiguous Allocation

b Variable size partition.



- ✓ No internal frag
- ✓ Degree of M.P. variable
- ✓ process size is not fixed.





✗ External fragmentation

↳ soln Compaction

❖Advantages:

- There are very less chances of **internal fragmentation** - Degree of multi-programming is not limited/fixed
- Size of the process is not also limited, any size process may get load into the main memory.

❖Disadvantages:

- **External fragmentation**: due to loading and removing of processes into and from the main memory, main memory gets fragmented i.e. it gets divided into used partitions and free partitions. In such case, if any new process is requesting for the memory and even if the requested size of memory is available, but due to unavailability of the memory in a contiguous manner request of that process cannot be accepted, this problem is referred as an external fragmentation.
- External fragmentation is the biggest problem under contiguous memory allocation, and hence these solve using compaction.
- **Compaction**
  - shuffling of main memory can be done in such a way that all used partitions can be shifted to one side and all free partitions can be shifted to other side and contiguous large free partition will be made available for the new processes.
  - Compaction is practically not feasible as there is need to do recalculations of addresses every time.

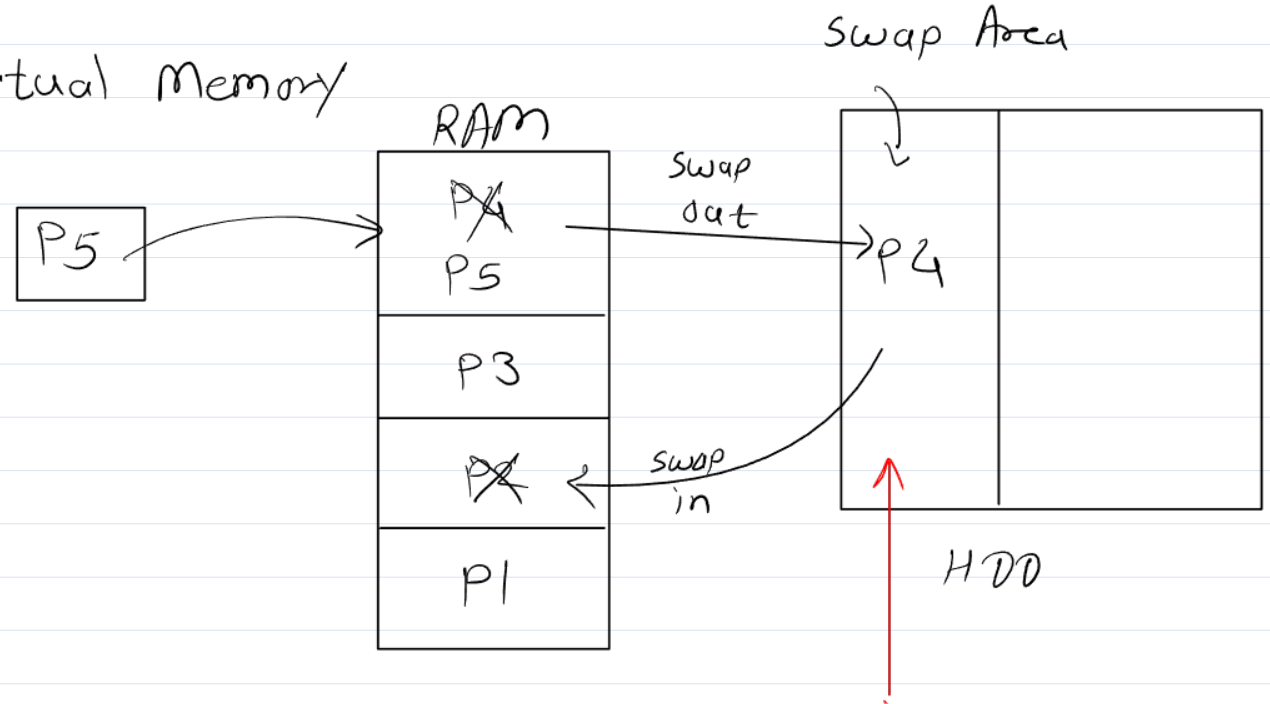
SUNBEAM

Virtual Memory

- The portion of the hard disk which is used by OS as an extension of RAM, is called as "**virtual memory".**
- If sufficient RAM is not available to execute a new program or grow existing process, then some of the inactive process is shifted from main memory (RAM), so that new program can execute in RAM (or existing process can grow).
- It is also called as "**swap area" or "swap space".**
- Shifting a process from RAM to swap area is called as "swap out" and shifting a process from swap to RAM is called as "swap in".
- In few OS, swap area is created in form of a partition. E.g. UNIX, Linux, ..
- In few OS, swap area is created in form of a file E.g. Windows (pagefile.sys), ..
- **Virtual memory advantages:**
  - Can execute more number of programs.
  - Can execute bigger sized programs.

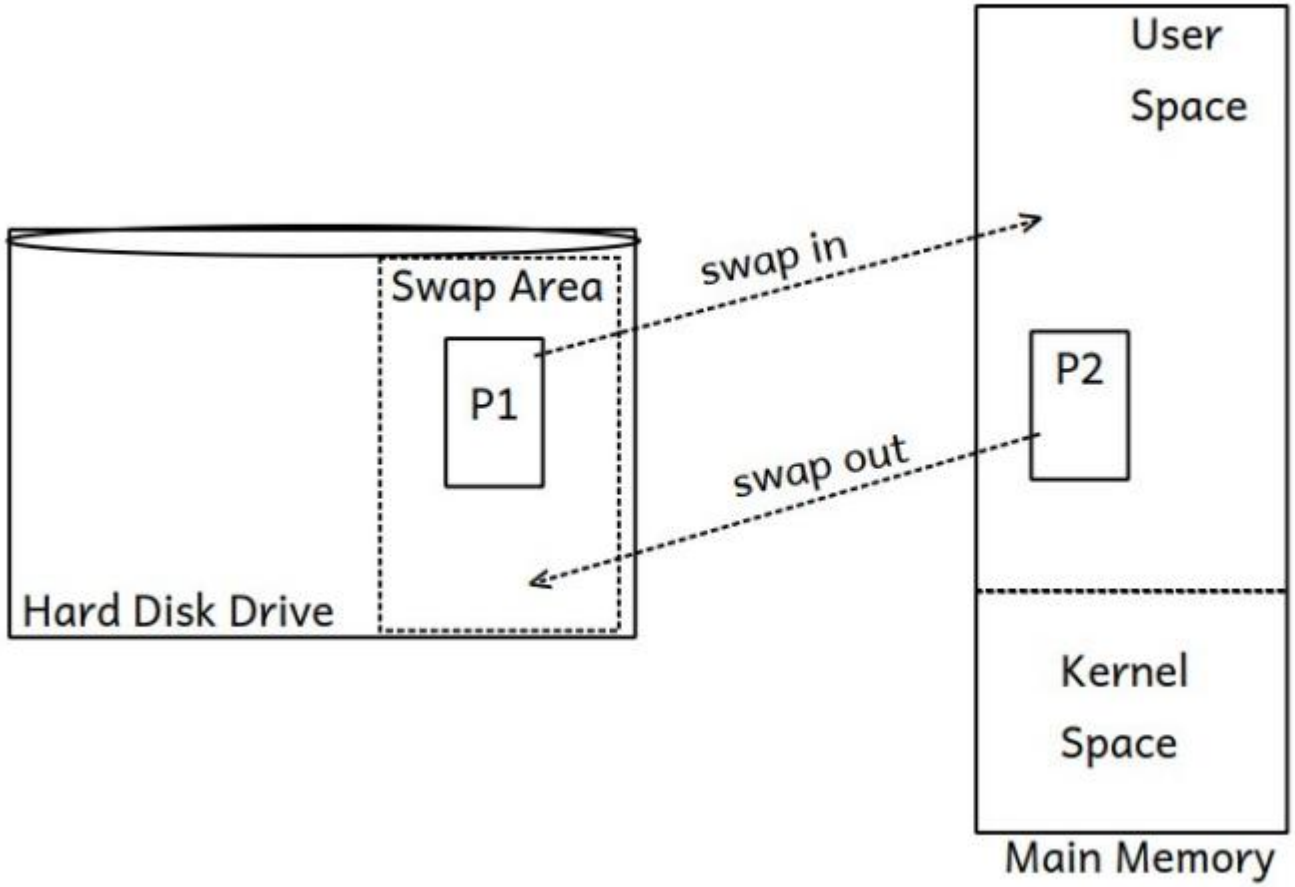
SUNBEAM

* Virtual Memory



- these is not actual RAM it formatted like RAM.
- To Keep Inactive Running program.

SWAPPING: MEMORY MANAGER



❖ Non- contiguous Memory Allocation:

- Under this method, process can complete its execution even if memory gets allocated for it in a non - contiguous manner, and it can be achieved by two memory management techniques:

    1. Segmentation

    2. Paging

- So by using segmentation & paging techniques, process can complete its execution even after memory gets allocated for it in a **non- contiguous manner.**

SUNBEAM

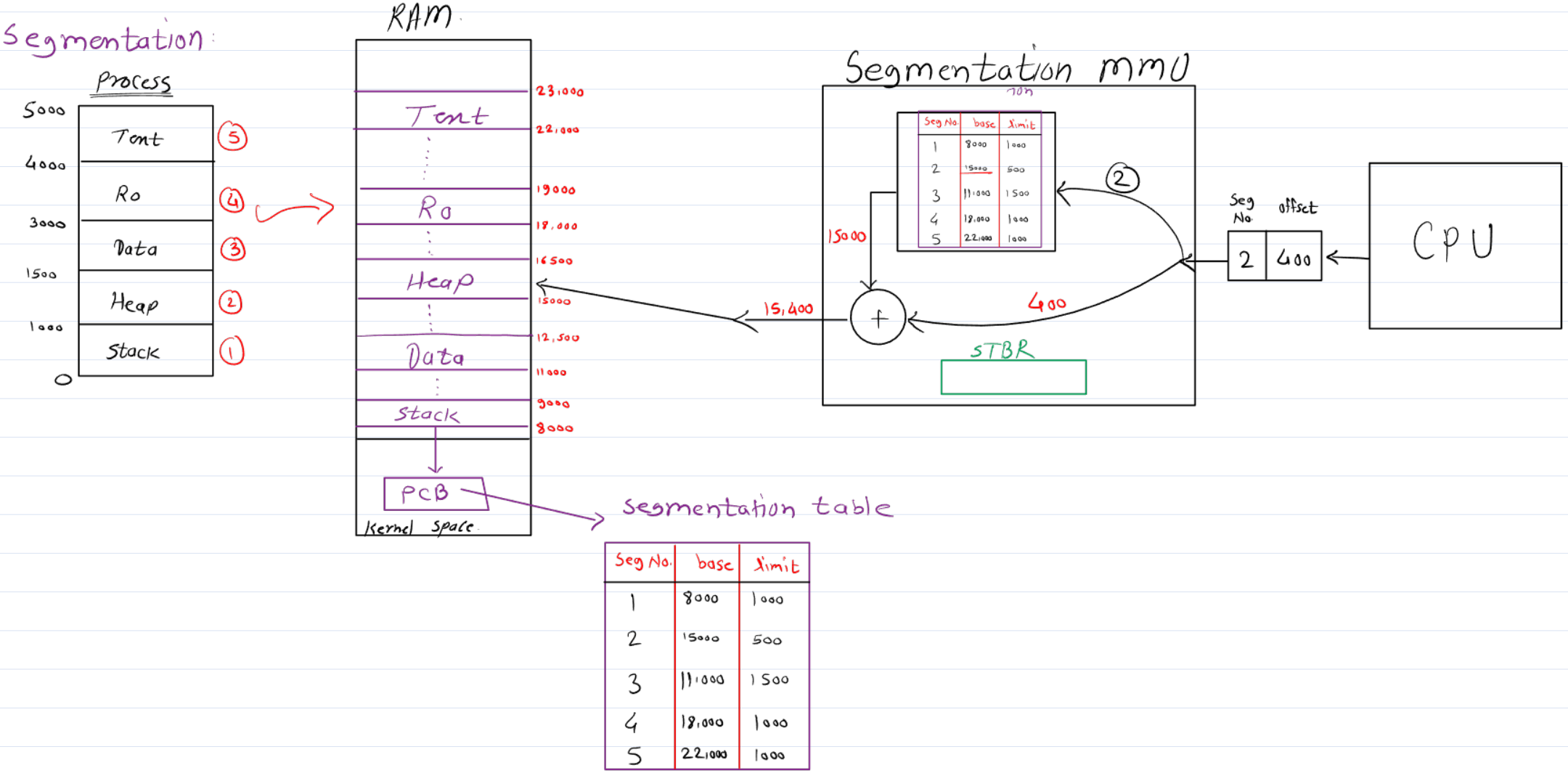
1. Segmentation

- Instead of allocating contiguous memory for the whole process, contiguous memory for each segment can be allocated. This scheme is known as "s**egmentation**".
- Since process does not need contiguous memory for entire process, external fragmentation will be reduced.
- In this scheme, PCB is associated with a segment table which contains base and limit (size) of each segment of the process. During context switch these values will be loaded into MMU segment table.
- CPU request virtual address in form of segment address and offset address.
- Based on segment address appropriate base-limit pair from MMU is used to calculate physical address as shown in diag.
- MMU also contains STBR (Segment Table Base Register) register which contains address of current process's segment table in the RAM.

SUNBEAM

* Non - Contigious Memory Allocation.

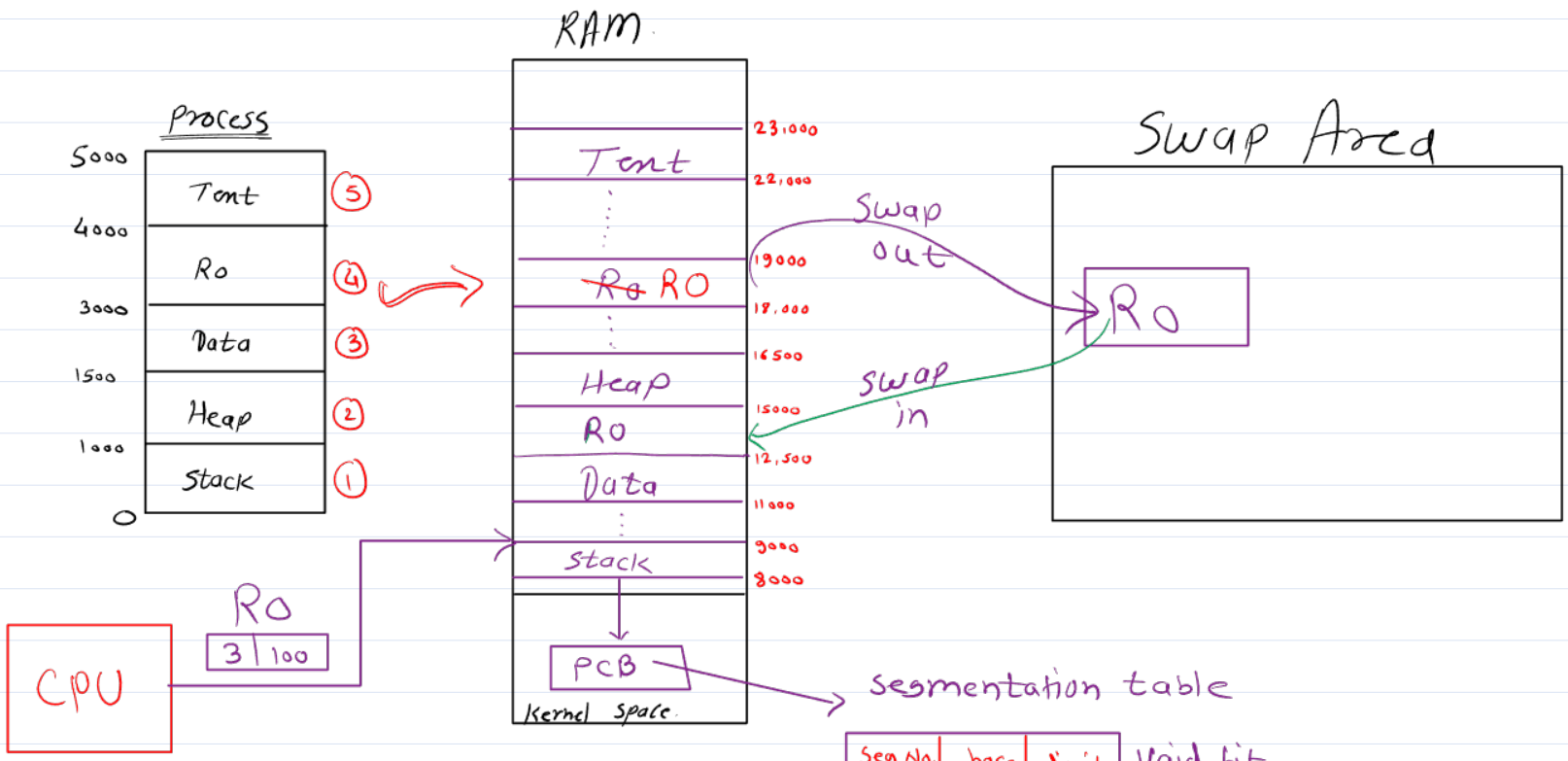
(1) Segmentation:



Segmentation table

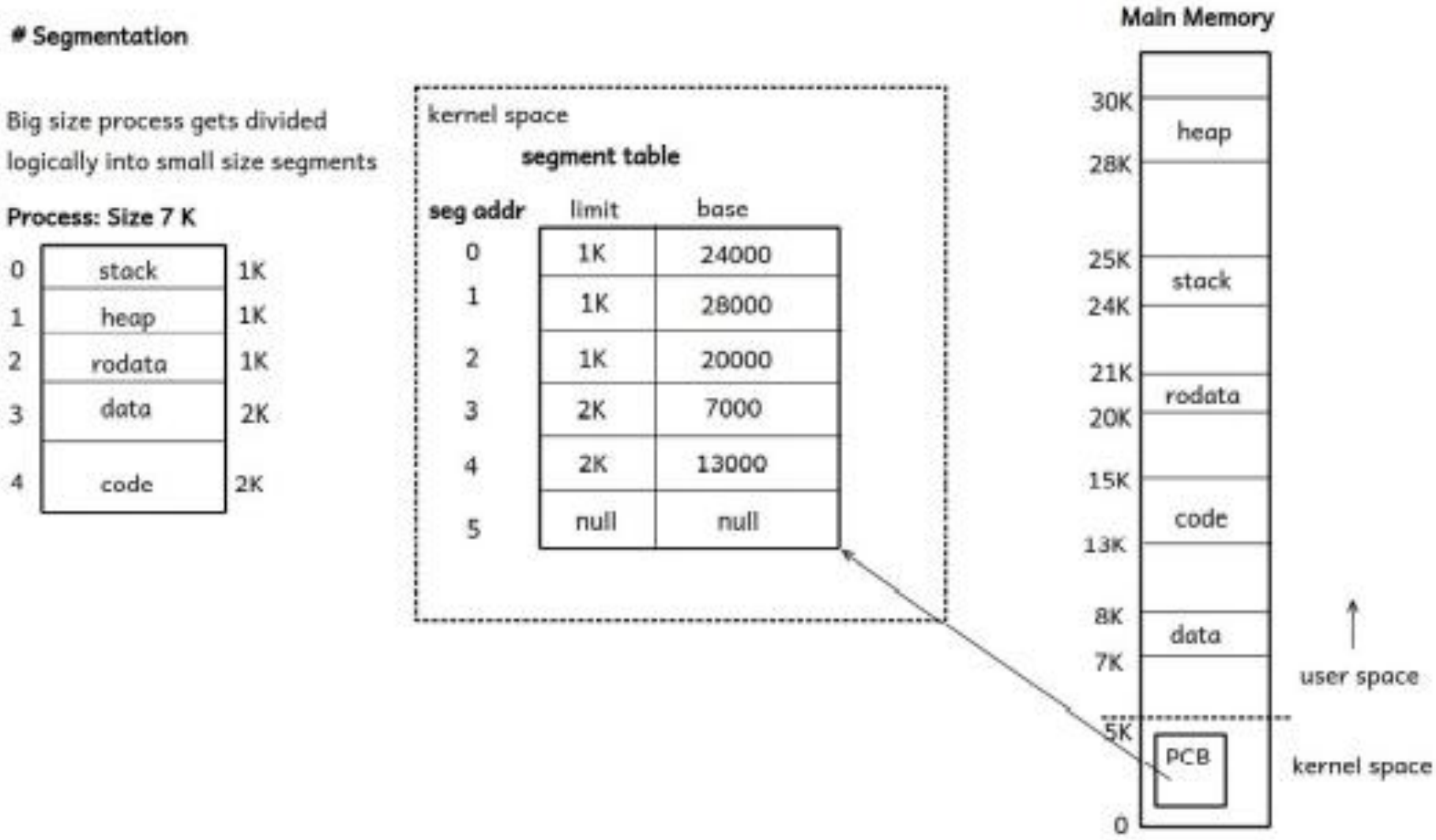
Seg No.	base	limit
1	8000	1000
2	15000	500
3	11,000	1500
4	18,000	1000
5	22,000	1000

* Segmentation & virtual memory.



Segmentation table

Seg No.	base	limit	Valid bit
1	8000	1000	1
2	15000	500	1
3	11,000	1500	0
4	18,000	1000	1
5	22,000	1000	1


Segmentation
Big size process gets divided logically into small size segments
Process: Size 7 K
0 stack 1K
1 heap 1K
2 rodata 1K
3 data 2K
4 code 2K
kernel space
segment table
seg addr limit base
0 1K 24000
1 1K 28000
2 1K 20000
3 2K 7000
4 2K 13000
5 null null
Main Memory
30K
heap
28K
25K
stack
24K
21K
rodata
20K
15K
code
13K
8K
data
7K
5K
PCB
0
user space
kernel space

❖Demand Segmentation

- If virtual memory concept is used along with segmentation scheme, in case low memory, OS may swap out a segment of inactive process.
- When that process again start executing and ask for same segment (swapped out), the segment will be loaded back in the RAM. This is called as "**demand segmentation**".
- Each entry of the segment table contains base & limit of a segment. It also contains additional bits like segment permissions, valid bit, dirty bit, etc
- If segment is present in main memory, its entry in segment table is said to be valid (v=1).
- If segment is swapped out, its entry in segment table is said to be invalid (v=0).

SUNBEAM

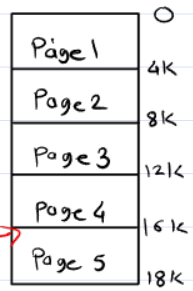
2. Paging :

- In this technique, physical memory (i.e. user space of a main memory) is divided into fixed size of blocks referred as **frames**, and process's logical memory space is divided into same size of blocks referred as **pages,** whereas maximum size of page must be equal to size of frame, i.e.
- **if e.g. size of frame = 4K, then maximum size of each page must be 4K, size of page may be less than 4K.**
- As process is divided into pages, so when it is requesting for memory, pages of one process may gets loaded into the main memory at any free frames, and for a process memory gets allocated in a non - contiguous manner.
- One page is allocated to one empty frame.
- OS keep track of free frames in form of a linked list
- There is no external fragmentation in paging.
- Internal fragmentation may exists in paging when the size of page is less than size of frame.

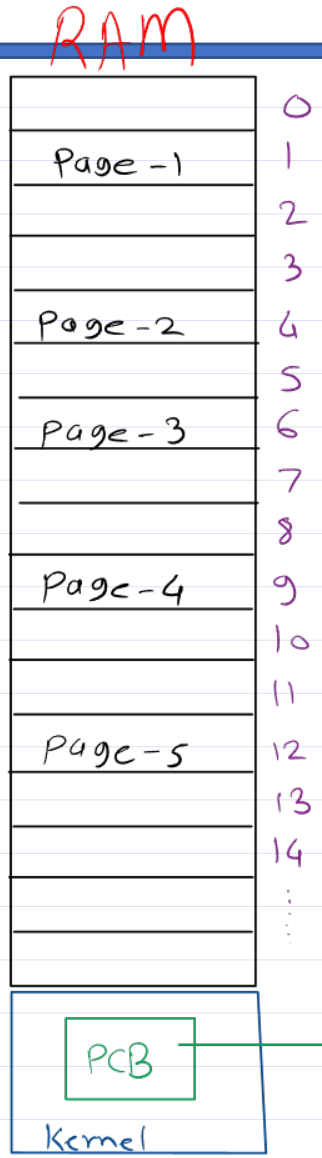
SUNBEAM

* Paging

Process (18Kb)



process divide small size logical partition called as "Page" / logical page.

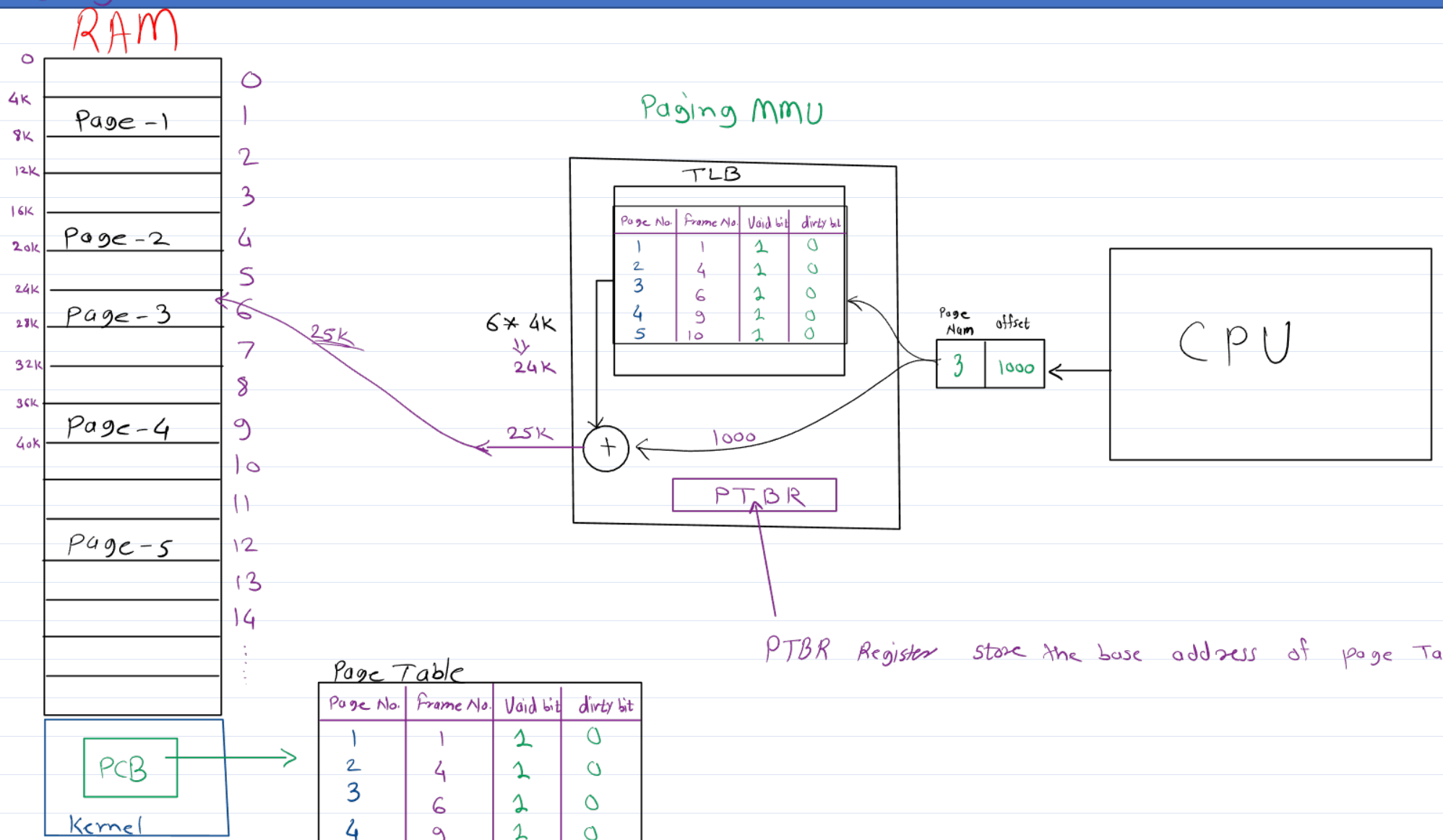


- Ram divided in small fix size partition called as frame / physical page
- Linux x86 arch ⇒ frame size 4Kb.

Page Table

Page No.	Frame No.	Valid bit	dirty bit
1	1	1	0
2	4	1	0
3	6	1	0
4	9	1	0
5	10	1	0

*Paging MMU


RAM
0
4K
8K
12K
16K
20K
24K
28K
32K
36K
40K
Page-1
Page-2
Page-3
Page-4
Page-5
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
PCB
Kernel
Paging MMU
TLB
Page No. | Frame No. | Valid bit | dirty bit
1 | 1 | 1 | 0
2 | 4 | 1 | 0
3 | 6 | 1 | 0
4 | 9 | 1 | 0
5 | 10 | 1 | 0
6 * 4K
24K
25K
25K
1000
Page Num
offset
3
1000
CPU
PTBR
PTBR Register store the base address of page Table.
Page Table
Page No. | Frame No. | Valid bit | dirty bit
1 | 1 | 1 | 0
2 | 4 | 1 | 0
3 | 6 | 1 | 0
4 | 9 | 1 | 0
5 | 10 | 1 | 0

- As pages of a one process gets loaded randomly into the main memory, and in a system thousands processes are running at a time, so to keep track on all the pages of each process, an OS maintains one table per process referred as **a page table** in which information about all the pages of that process can be kept.
- During context switch this table is loaded into MMU.
- CPU requests a virtual address in form of page address and offset address.
- It will be converted into physical address as shown in diag. MMU also contains a PTBR, which keeps address of page table in RAM.
- Frame size can be configured in the hardware. It can be 1KB, 2KB or 4KB, ...
- **Page table entry**
  - Each PTE is of 32-bit (on x86 arch) and it contains
    - Frame address
    - Permissions (read or write)
    - Validity bit
    - Dirty bit ...

SUNBEAM

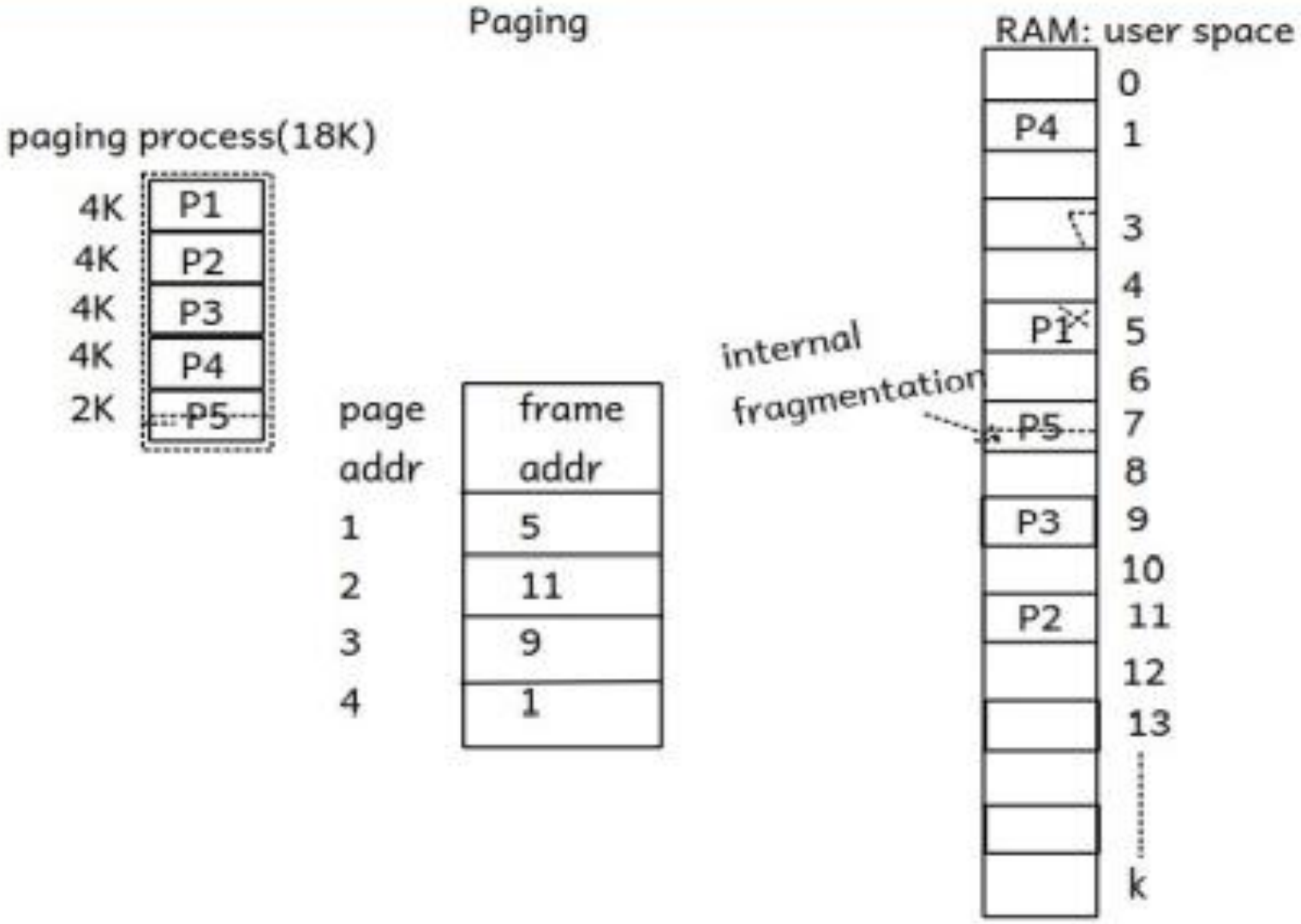
- **TLB (Translation Look-Aside Buffer)**
- Cache TLB is high-speed associative cache memory used for address translation in paging MMU.
- TLB has limited entries (e.g. in P6 arch TLB has 32 entries) storing recently translated page address and frame address mappings.
- The page address given by CPU, will be compared at once with all the entries in TLB and corresponding frame address is found.
- If frame address is found (TLB hit), then it is used to calculate actual physical address in RAM .
- If frame address is not found (TLB miss), then PTBR (Page Table Base Register)is used to access actual page table of the process in the RAM (associated with PCB).
- Then page-frame address mapping is copied into TLB and thus physical address is calculated.
- If CPU requests for the same page again, its address will be found in the TLB and translation will be faster.

SUNBEAM

❖Demand Paging

- When virtual memory is used with paging memory management, pages can be swapped out in case of low memory.
- The pages will be loaded into main memory, when they are requested by the CPU. This is called as "demand paging".
  - Swapped out pages
  - Pages from program image (executable file on disk)
  - Dynamically allocated pages
- **Virtual pages vs Logical pages**
  - By default all pages of user space process can be swapped out/in.
  - This may change physical address of the page. All such pages whose physical address may change are referred as "Virtual pages".
  - Few kernel pages are never swapped out.
  - So their physical address remains same forever. All such pages whose physical address will not change are referred as "Logical pages"

SUNBEAM

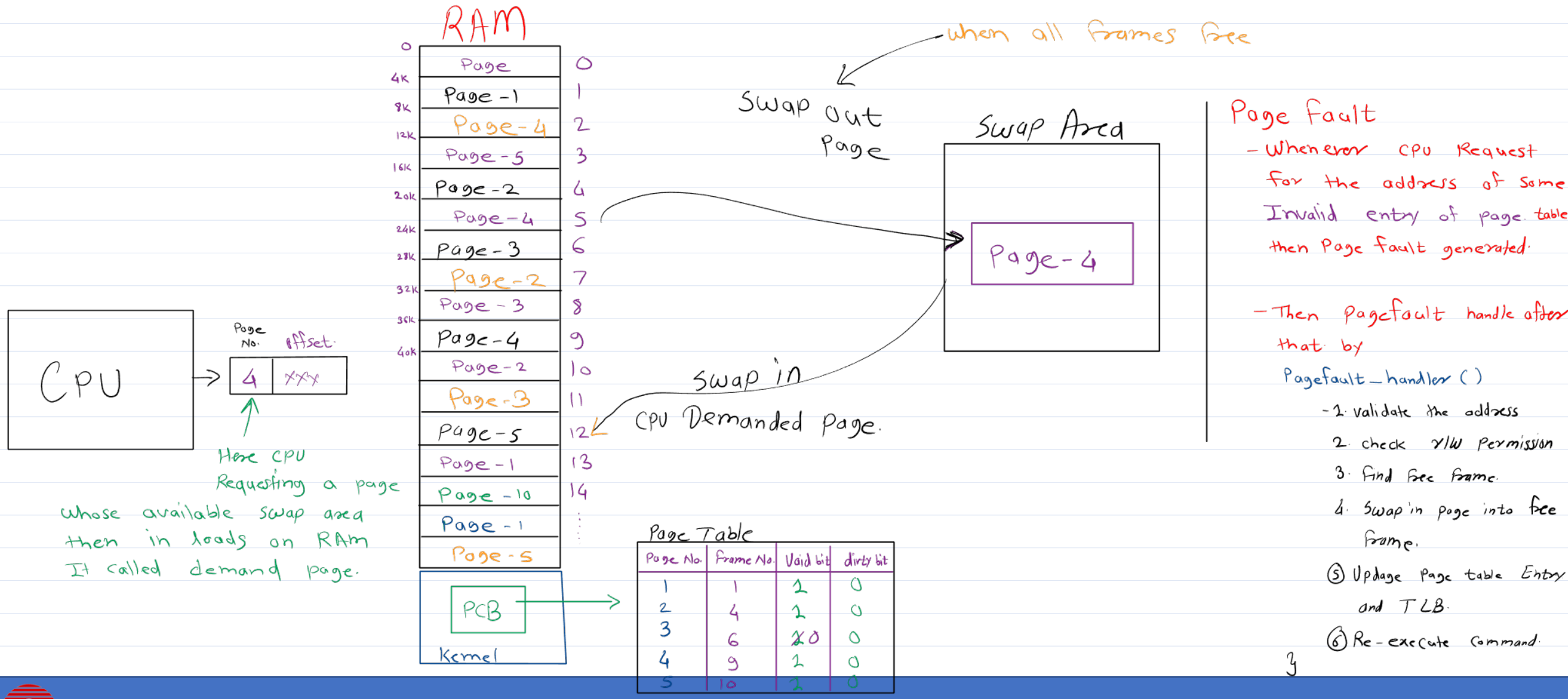

Paging
paging process(18K)
4K P1
4K P2
4K P3
4K P4
2K P5
page addr
frame addr
1 5
2 11
3 9
4 1
RAM: user space
internal fragmentation
0
1
3
4
5
6
7
8
9
10
11
12
13
k
P4
P1
P5
P3
P2

Page Fault

- Each page table entry contains frame address, permissions, dirty bit, valid bit, etc.
- If page is present in main memory its page table entry is **valid** (**valid bit = 1**).
- If page is not present in main memory, its page table entry is **not valid (valid bit = 0).**
- This is possible due to one of the following reasons:
  - Page address is not valid (dangling pointer).
  - Page is on disk/swapped out.
- **If CPU requests a page that is not present in main memory (i.e. page table entry valid bit=0), then "page fault" occurs.**

SUNBEAM

Paging + Virtual memory



Page Fault

- Whenever CPU Request for the address of some Invalid entry of page table then Page fault generated.

- Then Pagefault handle after that by

Pagefault_handler()

- 1. validate the address
2. check r/w Permission
3. Find free frame.
4. Swap in page into free frame.
5. Update Page table Entry and TLB.
6. Re-execute Command.

}

- **Then OS's page fault exception handler is invoked, which handles page faults as follows:**
  1. Check virtual address due to which page fault occured. If it is not valid (i.e. dangling pointer), terminate the process. (Validity fault).
  2. Check if read-write operation is permitted on the address. If not, terminate the process . (Protection fault).
  3. If virtual address is valid (i.e. page is swapped out), then locate one empty frame in the RAM.
  4. If page is on swap device or hard disk, **swap in** the page in that **frame**.
  5. Update **page table entry** i.e. add new frame address and valid bit = 1 into PTE.
  6. Restart the instruction for which page fault occurred.

SUNBEAM

Page Replacement Algorithms

- While handling page fault if no empty frame found (step 3), then some page of any process need to be swapped out. This page is called as "**victim" page**.
- The algorithm used to decide the victim page is called as "**page replacement algorithm".**
- **There are three important page replacement algorithms.**
  1. **FIFO**
  2. **Optimal**
  3. **LRU**

SUNBEAM

1. FIFO

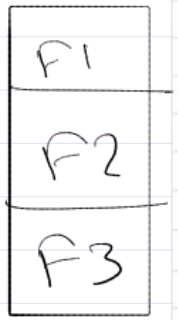
- The page brought in memory first, will be swapped out first.
- Sometimes in this algorithm, if number of frames are increased, number of page faults also increase.
- This abnormal behaviour is called as **"Belady's Anomaly**“

Given :

I/P Page Sequence : 1 2 3 4 1 2 5 1 2 3 4 5

Number of Frames : 3

FIFO Page Replacement



Page Request	1	2	3	4	1	2	5	1	2	3	4	5
F1	1	1	1 →	4	4	4 →	5	5	5	5	5 ←	5
F2		2	2	2 →	1	1	1	1	1 →	3	3	3
F3			3	3	3 →	2	2	2	2	2 →	4	4
Page Fault / Hit	F	F	F	F	F	F	F	H	H	F	F	H

Number of Page Faults = 9 Number of Page Hits = 3

Does Belady's Anomaly occur?

2. OPTIMAL

- The page not required in near future is swapped out.
- This algorithm gives minimum number of page faults.
- This algorithm is not practically implementable.

3. LRU

- The page which not used for longer duration will be swapped out.
- This algorithm is used in most OS like Linux, Windows, ...
- LRU mechanism is implemented using "stack based approach" or "counter based approach".
- This makes algorithm implementation slower.
- **Approximate LRU algorithms** provide **nearly the same performance** as true LRU but are **simpler and faster** to implement in operating systems.

SUNBEAM

Given :

I/P Page Sequence : 1 2 3 4 1 2 5 1 2 3 4 5

Number of Frames : 3

Optimal Page Replacement

Page Request	1	2	3	4	1	2	5	1	2	3	4	5
F1	1	1	1	1	1	1	1	1	1 → 3	3	3	3
F2		2	2	2	2	2	2	2	2	2 → 4	4	4
F3			3 → 4	4	4	4 → 5	5	5	5	5	5	5
Page Fault / Hit	F	F	F	F	H	H	F	H	H	F	F	H

Number of Page Faults = 7 Number of Page Hits = 5

Given :

I/P Page Sequence : 1 2 3 4 1 2 5 1 2 3 4 5

Number of Frames : 3

LRU Page Replacement

Page Request	1	2	3	4	1	2	5	1	2	3	4	5
F1	1	1	1 →	4	4	4 →	5	5	5 →	3	3	3
F2		2	2	2 →	1	1	1	1	1	1 →	4	4
F3			3	3	3 →	2	2	2	2	2	2 →	5
Page Fault / Hit	F	F	F	F	F	F	F	H	H	F	F	F

Number of Page Faults = 10 Number of Page Hits = 2

- **Dirty Bit**
- Each entry in page table has a dirty bit.
- When page is swapped in, dirty bit is set to 0.
- When write operation is performed on any page, its dirty bit is set to 1.
- It indicate that copy of the page in RAM differ from the copy in swap area.
- When such page need to be swapped out again, OS check its dirty bit.
- If bit=0 (page is not modified) actual disk IO is skipped and improves performance of paging operation.
- If bit=1 (page is modified), page is physically overwritten on its older copy in the swap area.

SUNBEAM

➢Thrashing:

- If any process spends more time on paging rather than execution, then this high paging activity is referred as thrashing.
- If number of programs are running in comparatively smaller RAM, a lot of system time will be spent into page swapping (paging) activity.
- Due to this overall system performance is reduced.
- The problem can be solved by increasing RAM size in the machine

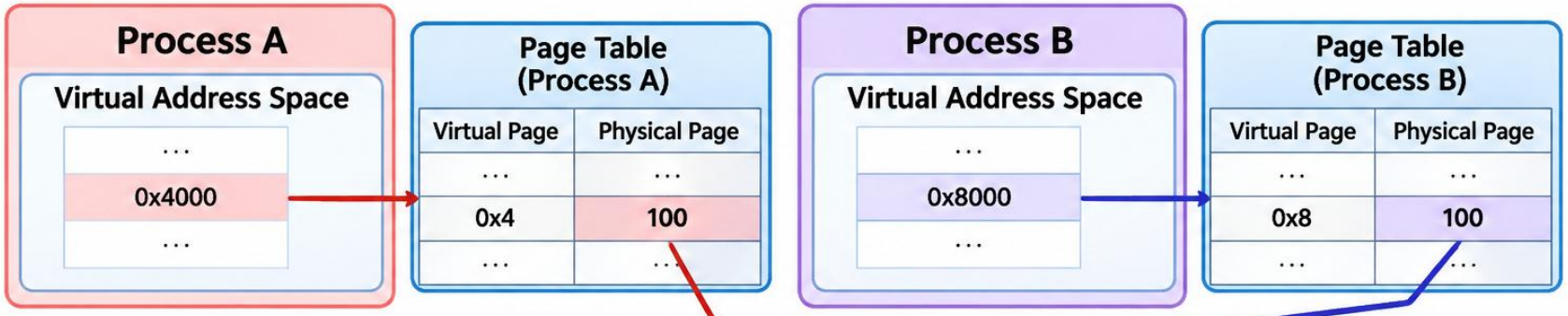
SUNBEAM

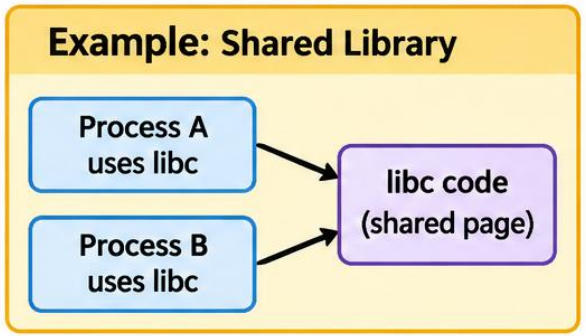
- **Shared Pages**
  - **Shared pages** are physical memory pages that can be used by **multiple processes**.
  - Each process has its own **virtual address space**, but their **page tables can point to the same physical page**.
  - Shared pages are commonly used for:
    - **Shared libraries** such as libc
    - **Shared memory** for IPC
    - **Read-only code/data**
  - **Advantage:** Saves RAM because the same physical page does not need to be loaded multiple times.
  - **Example:** If 10 processes use the same libc library, they can share the same physical memory pages containing its read-only code.

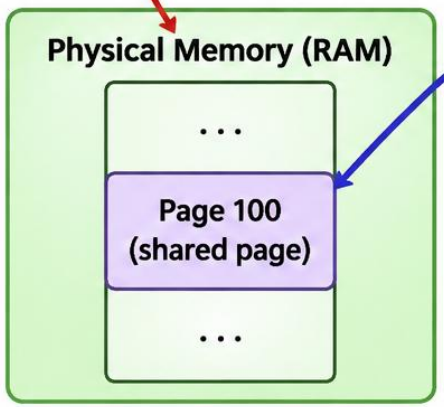
SUNBEAM

Shared Pages

A single physical memory page can be mapped to the address spaces of multiple processes.



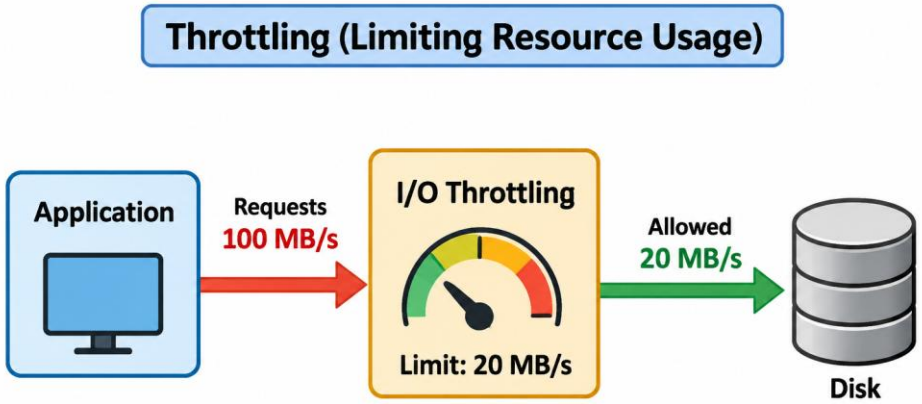




Key Points

- Each process has its own virtual address space.
- Page tables can point to the same physical page.
- Used for shared libraries, shared memory (IPC), read-only code.
- Saves physical memory by avoiding duplicate copies.
- Same physical page can be used by multiple processes.

- **Throttling**
  - **Throttling** means intentionally **limiting the rate of resource usage** by a process or application.
  - It prevents one process from consuming **excessive system resources**.
  - Throttling can be applied to:
    - **CPU usage**
    - **Disk I/O**
    - **Network bandwidth**
    - Other system resources
  - Helps maintain **system performance and fairness** among processes.
  - Throttling = Intentionally limiting resource usage to control system load.



SUNBEAM



Thank you!

Kiran Jaybhave

email – kiran.jaybhave@sunbeaminfo.com